

Algorithms Efficiency Research
Data Structures and Algorithms 1
Robert Poley
5-1-26

Introduction

Throughout this paper I will provide the data gathered from testing a series of self implemented algorithms. We will be focused on the time taken to complete the searching and sorting of large sets of numerical data stored in Arrays as well as ArrayLists and the difference in time between these two data types. I will be testing search algorithms and their time complexity via the following methods: Linear, RecursiveLinear, LinearBinary, And BinaryTree. Then I will evaluate the time complexity of the following sorting algorithms: BubbleSort, InsertionSort, SelectionSort, ShellSort, QuickSort, and MergeSort.

This research has led me to renew previous testing methods, for instance the speed of running these algorithms on current hardware ruled out using methods such as milliseconds useless. This brought me to the discovery of using nanoseconds as a time measurement which much better suits the demands of this research. This project also led me to the discovery of the fact that the Java Just In Time Compiler takes time to reach full efficiency. Consistent results that deviated from expected outputs led me to further my research which found this is a well known and documented phenomenon. This will be seen throughout my graphs and discussed in more detail at a later point.

LinearSearch

To test the LinearSearch method, I began by creating a loop that generates Arrays of random numbers from 0-9999, the loop starts with 1,000 random values in the range and increases the size of the next Array by 1,000. Each run of this loop takes the current system time and then searches for a given arbitrary value "2563". After searching the array my software takes the system time again to get an idea of how long it is taking the system to parse through the array, then the output is decided based on whether or not the value was found and the time taken is always displayed. The process for the ArrayList version of this loop is primarily the same so rather than explaining that again I will skip to the time taken.

This process was insanely efficient with little to no time spent on either the Array or the ArrayList version of the process with the time spent being tracked in nanoseconds as milliseconds were not finite enough to track the time taken to gather meaningful data. As seen in the graph below, whether using an Array or an ArrayList to store the dataset, the time cost of this search method can be entirely luck based. At some points the algorithm was able to find a target value in a shorter amount of time than in smaller datasets.

Graph For LinearSearch



What is clear and rarely deviates from this rule is that using this search method with data stored in an ArrayList takes longer than using it with a typical array. While there are outliers this can be ruled out as a margin of error as the time cost of running this is partially dependent on luck as mentioned before.

BinaryTree Search

My methodology for testing the BinaryTree Search involves taking the largest given Array from the prior loops and converting it into a BinaryTree. I created a method that loops through the Array and adds each index to a newly created BinaryTree, we then use the BinaryTree's contains function in another method.

Much like with the linear search method, the BinaryTree Search was so fast it barely provided meaningful data but it is worth mentioning that the increased efficiency of the way a BinaryTree's contains method works can keep this time low as data sets expand versus a linear search, as half of the potential searches are eliminated with each step through the nodes.

BinaryTree Search Graph



This graph mainly provides insight on how inconsistent searching via Binary Trees can be. The way in which a Binary Tree is searched can make the time to complete heavily dependent on how many values are stored before the value we are looking for. Binary trees can be very useful in certain scenarios as they can remove up to half of the remaining possibilities upon each search, they may still take a very long time if the target value is buried.

LinearBinary Search

Although the LinearSearch was quick in the datasets provided we can take it a step further with a LinearBinary Search. This method of searching does require a sorted list to operate which can add some time to the operation depending on the size of the dataset, the graph below will provide a better idea as to how the size might affect the time taken to sort.

For the sorting of these datasets I used the BubbleSort method but any would work, this sorting method will be discussed later in the paper. Once the dataset is sorted the LinearBinary Search starts out by grabbing the middle index and checking to see if that value is the same, smaller, or larger than the target value. With this information the algorithm either stops or decides to move one direction through the dataset, cutting the possible traversal time in half.

Array: 50,000 Values

Time taken to complete bubble sort: 2613 (Milliseconds)

Swaps: 627,031,072

ArrayList: 50,000 Values

Time taken to complete bubble sort: 9386 (Milliseconds)

Swaps: 626,870,447

As seen in the above data, we use the BubbleSort method to provide an ordered dataset for the LinearBinary Search method to work with. It is worth noting that although the ArrayList implementation happened to take less swaps in this scenario, it still took significantly longer to accomplish the same task. If you are working with many large datasets and using a BinaryLinear Search, using an Array as opposed to an ArrayList could save lots of time.

LinearBinary Search Graph



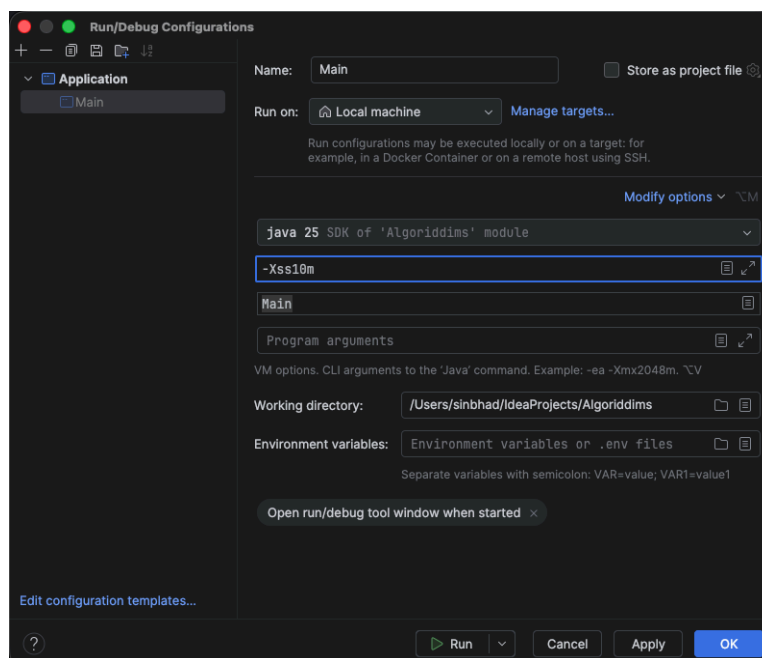
When compared to the graph of the time to complete searches via the LinearSearch algorithm, the LinearBinary Search algorithm trends remarkably lower, with the highest completion time being significantly faster. This graph also continues the trend of Array work being faster than ArrayList work with an added layer.

The fact that this search algorithm requires a sorted set of data means one needs to think carefully about the efficiency of the sorting algorithm used in unison with the LinearBinary Search. For example, in a worst case scenario, one might choose to use a BubbleSort algorithm on the dataset which greatly increases the time to finish the search, this is emphasized further when working with an ArrayList. My sort time was not included here as the main focus is the time of the search algorithm itself as opposed to the sorting algorithm, but there was a significant amount of time spent waiting for the datasets to be sorted as I unfortunately used a BubbleSort algorithm for this test.

RecursiveLinear Search

As the name implies the RecursiveLinear Search method primarily functions in the same manner as the LinearSearch with one key difference, it moves along the dataset recursively as opposed to using a loop. This approach slows down the process but allows for a much simpler implementation as all you need to program is a base case (to prevent infinite recursion) , a case that returns true and then calls the function again, increasing the index parameter each time.

Recursively calling the function has a much higher time complexity than looping through all the values so even with a very large unordered dataset, the time to complete the search stays relatively quick, though not as fast as iteratively looping the data. One other important note is that as we are testing very large datasets with recursion, we can run into the wall of recursion's predefined limits, causing the IDE to throw a stack overflow. To prevent stack overflows on massive datasets we can expand the memory pool allocated by the IDE. I started by increasing the limit from four megabytes to ten and after a few demos I did not see any issues so I will leave it there for the purposes of this demonstration. Below is an image to show how this was accomplished.



Graph for LinearBinary Search



The data from this chart does not paint a good picture of how the timing may differ using this method on an Array of data as opposed to an ArrayList as the data tested is randomly generated. As mentioned in the section on a typical LinearSearch algorithm, the timing here is partially dependent on luck as the ordering of values has the potential to differ massively. To circumvent this and get a more accurate analysis, I could have run the algorithm on one the dataset stored in an Array, converted it to an ArrayList and then ran the algorithm on that for consistency in testing. For the sake of time and purposes of this research we will not be doing so.

Bubble Sort

The BubbleSort method is one of the simplest sorting methods tested in this research, it is a nested loop that iterates through a dataset iteratively, comparing each index to the next in the line and swapping them if the index to the left is larger. The algorithm does this repeatedly until there are no more steps left, at which point the dataset is guaranteed to be sorted. This method can take many steps to complete and the time taken can grow very quickly as seen in my testing.

As seen in the data below, the time to complete the sorting of a dataset can be heavily affected by the data type you are using, with the dataset stored in an Array consistently taking significantly less time to complete. While the swaps taken to complete the sorting of these datasets is remotely unaffected by the data type used to store the dataset, this can still be a valuable metric to track as it can give insight to how efficient one sorting method is compared to another in terms other than just time so if your test sets are not enough to stress your algorithm, you can still get a rough idea of how it might work with a substantially larger set.

Graphs for BubbleSort



Insertion Sort

The InsertionSort algorithm works by comparing and shifting given values in a dataset. It starts with the first element of an Array or ArrayList and compares the value to the right, swapping them if the next value is smaller than the current. The values on their left are their own Array or ArrayList now and as this temporary dataset expands the next value gets compared to each value in the temporary set and gets placed where it should belong. Once the entire dataset has been evaluated the temporary dataset gets reassigned to the original dataset.

Ordering here plays a much smaller role here for time to complete the sort than with algorithms such as BubbleSort as the algorithm will only ever do one pass of the whole list. While backtracking the past data can be somewhat time consuming on larger datasets, this algorithm's time to complete took at most, less than ten percent of the time to complete as opposed to the BubbleSort algorithm.

Graph for InsertionSort



While ordering can affect the time to complete this sorting method it is very clear that using this algorithm with an Array is significantly faster than using it with an ArrayList.

Selection Sort

The SelectionSort algorithm works in a similar manner to the insertions sort with the key difference here being that the algorithm preemptively parses the unsorted section of the dataset to look for the smallest value, then it places the value where it belongs among the already sorted values. This method of evaluation does increase the time spent to sort a dataset but the main benefit to using this algorithm over an insertion sort is that the SelectionSort algorithm is more memory efficient.

When time is the biggest factor in deciding a sorting algorithm, an InsertionSort would be ideal but if you have major memory constraints the SelectionSort algorithm would be an ideal replacement.

Graph for SelectionSort



As seen in this chart the SelectionSort algorithm takes an average of 60% longer to finish sorting as opposed to the InsertionSort Algorithm.

Shell Sort

The ShellSort algorithm acts as an optimized implementation of an insertion sort, taking an already efficient algorithm a step further by using a divide and conquer approach. The dataset is segmented into smaller sections by a predetermined amount on each run, for the purposes of my testing I chose a divided of four for each step. These segments are sorted by an insertion sort and then segmented again until they have a gap of one value, once they have been sorted to their minimum value they are all joined together in an organized fashion.

Compared to previous sorting algorithms the efficiency of this algorithm is unmatched which you will see in the graph below. When paired with a search algorithm such as the LinearBinary Search which requires a sorted dataset, run times will significantly decrease.

ShellSort Graph



As seen in this data, the time to complete the sorting of datasets of the same size of the BubbleSort algorithm go from hundreds of milliseconds to less than thirty. While this may not seem significant, as datasets grow the time to complete these sorts stays consistently low, running this repeatedly with massive datasets can save immeasurable amounts of time.

This chart also provides more insight as to the efficiency of Arrays as compared to ArrayLists with the average ArrayList sort taking two to two and a half times as long to complete a opposed to an Array of the same size.

Merge Sort

The MergeSort algorithm also uses a divide and conquer approach similar to the ShellSort algorithm though in this algorithm, the dataset is broken up into segments until each segment contains exactly one element. From there the algorithm compares these smaller arrays and combines them, being that they are already sorted segments, the algorithm is only required to evaluate the first value of each segment and arranges them based on the comparison. This operation continues until all of the segments once more form one large dataset.

While this algorithm is more efficient than a shell sort in terms of time, it does use more memory so again, if memory is a limiting factor this algorithm may not be an optimal choice as opposed to other viable algorithms such as a shell sort.

MergeSort Graph



While the first run of this algorithm shows a large difference in time between the Array and the ArrayList, subsequent rows show a trend of the ArrayList counterpart of this operation taking roughly twice as long to complete as the Array operation.

Quick Sort

The quick sort algorithm works much like the MergeSort algorithm with the key difference being that this algorithm sorts the data upon a pivot point on the original Array or ArrayList rather than creating a bunch of tiny data collections. This operation sorts the left and the right of the pivot point and reduces the pivot point until the pivot is the last remaining element of the segment.

Using a pivot point and working through the original Array or ArrayList keeps the time cost low while also saving memory, providing a highly optimized sorting algorithm for almost any scenario.

Quick Sort Graph



From this data, we can again write off the high value at the beginning as a margin of error outlier, though it is still important to take note of, the reasoning for this will be discussed in the next section.

Looking at this graph, it may appear that the time cost is slightly higher than the previous graph, though they are very similar. The graph is skewed as the relatively long first run of the previously shrunk algorithm shrunk the rest of the data. The time to complete a sort of an ArrayList dataset took on average three to three and a half times longer to complete than that of a sort on an Array.

Key Findings

Time cost: While the time to search was affected by the ordering of the dataset, the trend that ArrayLists take longer to operate on than Arrays was made evident by the findings when comparing sort times for the given datasets. In every tested sort algorithm, the time taken to complete an ArrayList sort was anywhere from two to three times as long to complete.

JIT Compiler warmup time: As seen in the charts throughout this study, the first one to three runs consistently took significantly longer to finish than the rest of the operations. This was very perplexing to see as one would expect these values to be lower being that they are the smaller datasets. I did some unintended research to understand this anomaly and came across a phenomenon known as JIT Compiler warmup time. When running a Java program, the instructions do not run at optimal efficiency as they are first interpreted and then translated into machine code. By the time the instructions are completely translated into machine code the operation is able to run at complete efficiency. There are ways around this but without creating an intricate solution, it may be wise to run multiple datasets in a row if you are using the same algorithm on each set as opposed to running them one at a time so that you can take advantage of the most efficient state of the compiler.

Works Cited, Credits to classmates

[Recursive Linear Search](#)

[Shell Sort Resource 1](#)

[Shell Sort Resource 2](#)

[Quick Sort Resource 1](#)

[Quick Sort Resource 2](#)

[Merge Sort](#)

[JIT Warmup](#)

Andrew Davitt - Gave me tips to improve the efficiency of my bubble sort

Timothy Le - General thoughts on some of my implementation tactics mainly in my test cases